ソケット通信に着目したIoTファームウェア解析フレームワークの設計と予備的評価

東 政澄[1,a)]　掛井 将平[1]　齋藤 彰一[1]

概要：IoT 機器に搭載されるファームウェアは，更新頻度の低さや実行環境の制約により，汎用計算機と比べて脆弱な場合が多く，攻撃の踏み台として悪用される可能性が高い．この問題に対し，自動解析によって脆弱性を特定する研究が進められている．一方，これらの機器では，Web サーバーや設定管理機能などの複数プロセスが協調して動作し，プロセス間通信を介してデータの受け渡しが行われる．このため，単一プロセスのみを対象とした解析では，通信を介した処理の追跡が不十分となる．しかし従来手法には，プロセス間通信を十分に考慮していない手法や，通信関係を網羅的に解析するため解析コストが大きい手法があり，機器固有の性質を活用した効率的な解析は困難である．本研究では，プロセス間通信の一種であるソケット通信を考慮した，ファームウェアを自動解析する手法を提案する．本手法は，ファームウェアのバイナリを事前解析してソケット通信に関与するバイナリを抽出し，当該対象に対する詳細解析により効率的にプロセスを追跡する．評価では，提案手法のオーバーヘッドおよび実世界のファームウェアデータセットにおける性能を検証する．

キーワード：IoT ファームウェア, ソケット通信, 自動解析

Design and Preliminary Evaluation of an IoT Firmware Analysis Framework Focusing on Socket Communication

Masazumi Azuma[1,a)]　Shohei Kakei[1]　Shoichi Saito[1]

Abstract: IoT firmware is often vulnerable because of infrequent updates and constrained execution environments, making compromised devices potential footholds for further attacks. Although automated firmware analysis has been widely studied, many existing approaches focus on individual processes and may fail to trace data flows across inter-process communication. Other approaches analyze communication relationships exhaustively, resulting in high analysis costs. We therefore propose an automated firmware analysis method that focuses on socket communication. After unpacking the firmware, the method identifies binaries involved in socket communication through preliminary analysis and performs detailed analysis only on the selected binaries. This enables efficient tracking of interactions across processes. We evaluate the overhead and effectiveness of the proposed method using real-world firmware datasets.

Keywords: IoT Firmware, Socket Communication, Automated Analysis

1. はじめに

モノをインターネットに接続することで遠隔監視や制御を可能とする Internet of Things (IoT) は，その利便性から多岐にわたる用途で使われている．一方，IoT 機器の普及に伴い，これらの機器を標的とする攻撃も増加している．2016 年には，IoT マルウェア Mirai が多数の IoT 機器を侵害してボットネットを形成し，DNS サービス事業者 Dyn に対する大規模な DDoS 攻撃が発生した [1]．また，2024 年には，既知脆弱性を悪用して SOHO ルーター，IP カメ

[1] 名古屋工業大学
Nagoya Institute of Technology
[a)] m.azuma.818@nitech.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

ラ，DVR および NAS 等のインターネット接続機器を侵害し，侵入活動の発信源を秘匿するためのボットネットを構築していたことが確認された [2].

IoT 機器が攻撃対象として選ばれやすい要因として，リソース制約により十分なセキュリティ対策を実装しにくいことや，セキュリティアップデートが継続的に提供または適用されず，重大な脆弱性が長期間残存しやすいことが挙げられる [3]．さらに，実際の解析においてもファームウェアには多数の実行可能バイナリやライブラリ，設定ファイルなどが含まれており，専門家による手動解析には多くの時間と知識を要する．この問題に対し，ファームウェアの脆弱性を自動的に特定するための研究が進められている．

既存の自動解析手法の一部は，単一のバイナリまたはプロセスを解析対象とし，その内部で生じるデータフローを追跡する [4], [5]．一方，IoT 機器では，Web サーバーや設定管理デーモンなどの複数のプロセスが協調して動作し，ソケット通信や共有メモリなどのプロセス間通信 (Inter-Process Communication：IPC) を介してデータを受け渡す場合がある．したがって，単一プロセスのみを対象とする解析では，IPC を介して複数のプロセスに伝播する情報フローを追跡できず，複数プロセスが関与する脆弱性を見落とす可能性がある．

複数プロセス間の関係を考慮する手法として，ファームウェア全体から依存関係グラフを構築する静的解析 [6], [7] や，システム全体を対象とする動的テイント解析 [8], [9] が提案されている．しかし，前者は解析対象となるバイナリや探索経路の増加に伴って解析時間が増大し，後者はシステム全体のエミュレーションおよび命令単位の追跡を必要とするため，実行時オーバーヘッドが大きい．このため，実際に通信関係を持つプロセスへ解析対象を絞り込んだうえで，プロセス間の情報フローを追跡する手法が必要である．

そこで本研究では，同一機器内で行われるソケット通信に着目し，複数の解析を段階的に適用することで，動的テイント解析の対象を絞り込む解析機構を提案する．本手法では，まず，ファームウェア内のバイナリが持つ静的情報に基づき，ソケット通信を行う可能性のあるバイナリを抽出する．次に，抽出したバイナリを実行してソケット通信を観測し，実行時に観測されたエンドポイント情報に基づき，通信関係を持つ可能性のあるバイナリの組を候補として抽出する．最後に，特定したバイナリの組に対してのみ動的テイント解析を適用し，ソケット通信を介してプロセス間を伝播する情報フローを追跡する．これにより，ファームウェア全体に対して詳細な解析を一律に適用する場合と比較して，解析対象および解析負荷の削減を図る．

以下，第 2 章では，ファームウェアの自動解析に関する研究について述べる．第 3 章では，提案手法の設計および各解析処理について説明する．第 4 章では，提案手法の評価を行い，第 5 章で今後の課題を述べる．最後に，第 6 章で本稿をまとめる．

2. 関連研究

IoT ファームウェアに存在する脆弱性を特定するため，さまざまなアプローチによる自動解析手法が研究されている．これらの研究は，単一のコンポーネントを対象とする手法と，複数のコンポーネント間の関係を考慮する手法に大別される．本章では，単一コンポーネントを対象とする解析手法，複数コンポーネントに着目した静的解析手法，および複数コンポーネントに着目した動的解析手法について述べる．

2.1 単一コンポーネントを対象とする解析手法

プログラム内のデータフローを追跡する手法として，テイント解析が広く用いられている．例えば，Android アプリケーションを対象とする TaintDroid では，機密情報や外部入力がプログラム内をどのように伝播するかを動的に追跡する [10].

DTaint および EmTaint は，静的テイント解析によって単一のバイナリを解析し，外部入力から危険な関数に至るデータフローを追跡することで, 脆弱性を検出する [4], [11]. これらの手法は，個々のバイナリ内部におけるデータフローを効率的に解析できる一方，異なるプロセス間で行われる通信やデータの受け渡しを解析対象としていない．そのため，IPC を介して複数のプロセスに伝播するデータフローを追跡することは困難である．

2.2 複数コンポーネントに着目した静的解析

複数のコンポーネントを対象とする静的解析手法の一つとして, Karonte[6] が提案されている．Karonte は，ファームウェア全体を静的に解析してコンポーネント間の依存関係グラフを構築し，プロセス間にまたがるデータフローを推定することで，複数のコンポーネントが関与する脆弱性を検出する．一方，解析対象となるコンポーネントや依存関係が多い場合，依存関係グラフ上で探索すべき経路が増加し，解析時間が増大する．そのため，ファームウェアの構成によっては，現実的な時間内で解析を完了することが困難となる可能性がある．

この課題に対し，後続の研究では，Web インタフェースなどのフロントエンドから取得したキーワードを用いて，入力を処理するコンポーネントや関数を特定し，それらを起点として解析対象を絞り込む手法が提案されている [12]. また，大規模言語モデルなどを補助的に用いて，コードやデータの意味的な関係から入力処理箇所やコンポーネント間の関係を推定する手法も提案されている [7]．これらの手法により，解析対象の絞り込みや解析速度の改善が期待できる．一方，解析可能なアプリケーションや通信方式が限

© 2026 Information Processing Society of Japan

定されることや，外部から取得した意味情報またはモデルの出力に解析結果が依存するといった課題が残されている．

2.3 複数コンポーネントに着目した動的解析

ファームウェア全体を動的に解析するための基盤として，Firmadyne および AVATAR が提案されている [5], [13]．Firmadyne は，Linux ベースのファームウェアをエミュレーション環境上で起動し，ファームウェア全体の動的解析を可能にする．AVATAR は，エミュレーション環境と実機を組み合わせることで，周辺機器に依存するファームウェアの動的解析を支援する．これらの手法によってファームウェア全体の実行状態を観測できる一方，ファームウェアによってはエミュレーション環境上で正常に動作しない場合や，実機などの追加のハードウェアを必要とする場合がある．

複数のプロセスまたはホストを横断する情報フローを追跡する手法として，フルシステム動的テイント解析が挙げられる．フルシステム動的テイント解析は，OS カーネルや複数のプロセスを含むシステム全体を解析対象とし，実行中に伝播するテイント情報を追跡する技術である．RTAG および FlowDist では，分散システムを対象としてシステム全体の情報フローを追跡するとともに，アプリケーション層の情報を利用して解析対象を段階的に絞り込む手法が提案されている [8], [14]．

また，IoT ファームウェアを対象とする手法として，FDTAF や STAFF が提案されている [9], [15]．これらの手法は，フルシステム環境における実行情報およびテイント情報を利用することで，複数のプロセスやコンポーネントにまたがるデータの伝播を解析する．一方で，システム全体のエミュレーションと広範囲な命令追跡を必要とするため，解析対象とする命令数やプロセス数の増加に伴って，計算量および実行時オーバーヘッドが増大するという課題がある．

3. 提案手法

第 2 章で述べたように，ファームウェア全体を対象とする静的解析では，コンポーネント間の依存関係を網羅的に推定できる一方，解析対象となるコンポーネントや探索経路の増加に伴って解析時間が増大する．また，フルシステム動的テイント解析では，実行時に生じるプロセス間の情報フローを詳細に追跡できる一方，システム全体のエミュレーションおよび命令単位の追跡による実行時のオーバーヘッドが大きい．

そこで本研究では，ファームウェア全体に対して高コストな静的解析または動的テイント解析を一律に適用するのではなく，軽量な解析から詳細な解析へと段階的に処理を適用する．具体的には，バイナリの静的情報に基づくフィルタリングによってソケット通信を行う可能性のあるバイナリを抽出し，抽出したバイナリの実行を観測することで，実際に通信関係を形成するバイナリの組を特定する．その後，特定したバイナリの組に対してのみ動的テイント解析を適用する．これにより，全バイナリ間の依存関係を詳細に推定する静的解析と比較して解析対象となる関係を限定するとともに，システム全体を動的テイント解析する場合と比較して解析負荷を削減する．

提案手法の全体像を**図 1** に示す．本手法は，ソケット通信バイナリのフィルタリング，通信相手の特定，および動的テイント解析の三つの処理から構成される．まず，ファームウェアをアンパックし，得られたファイル群から，ソケット通信を行う可能性のあるバイナリを抽出する．次に，抽出したバイナリを解析環境上で実行し，ソケット通信の発生を観測することで，通信関係を形成するバイナリの組を特定する．最後に，特定したバイナリの組に対して動的テイント解析を適用し，ソケット通信を介してプロセス間を伝播する情報フローを追跡する．

3.1 適用範囲と制約

本研究で実装する解析機構は，解析環境上で実行可能なバイナリを対象とする．そのため，解析対象となるファイル形式，命令セットアーキテクチャ，およびバイナリの実行条件には，以下の制約がある．

第一に，本実装が直接の解析対象とするファイルは，動的リンク形式の ELF 実行可能バイナリに限定する．IoT ファームウェアには，ELF 実行可能バイナリのほか，シェルスクリプト，設定ファイル，共有ライブラリ，およびカーネルモジュールなど，多種多様なファイルが含まれている．本実装では，これらのうち，動的リンク形式の ELF 実行可能バイナリのみを動的テイント解析の対象とする．

第二に，本実装が対象とする命令セットアーキテクチャを，ARMv7 に限定する．ARMv7 は，既存の IoT ファームウェアで使用される命令セットアーキテクチャの中でも比較的高い割合を占めるため，本研究の解析対象として採用する．

第三に，本実装は，適切なコマンドライン引数を指定しなければ正常に起動できないバイナリを自動解析の対象外とする．このようなバイナリを解析するためには，必要となる引数の形式や値を事前に特定した上で実行する必要がある．引数を自動的に推定または生成するためには，静的解析やファジングなどを用いて引数の候補を探索する機構が必要となる．しかし，本実装はコマンドライン引数を推定または生成する機構を備えていない．このため，本研究では，引数を指定せずに起動可能なバイナリを自動解析の対象とする．

3.2 ソケット通信バイナリのフィルタリング

ファームウェアとして配布されているファイル群には，

© 2026 Information Processing Society of Japan

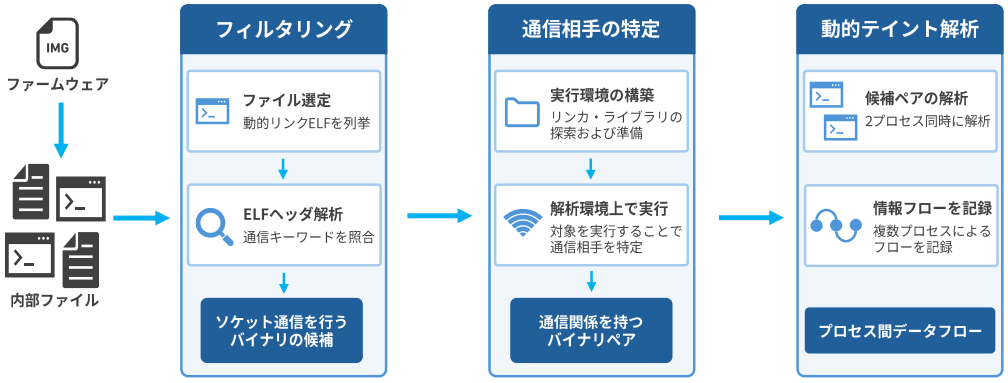


図 1　提案手法の概念図

Fig. 1　Overview of the Proposed Firmware Analysis Framework

ソケット通信に関与しないバイナリも多数含まれている．これらすべてを動的に実行して通信の有無を確認すると，解析時間が増大する．そこで本手法では，バイナリが持つ静的情報に基づいて，ソケット通信を行う可能性のあるバイナリを事前に抽出する．なお，本処理は実際のソケット通信の発生を判定するものではなく，後続の動的解析に与える候補バイナリを絞り込むことを目的とする．

まず，ファームウェアの各ファイルシステムを再帰的に走査し，通常ファイルを列挙する．このとき，シンボリックリンクは解析対象から除外する．また，ファイル名が".so"で終わるもの，またはファイル名に".so."を含むものは共有ライブラリとみなし，直接の解析対象から除外する．

次に，抽出した ELF バイナリから PT_DYNAMIC セグメントを取得する．PT_DYNAMIC セグメントが存在しないバイナリは，動的リンク情報を取得できないため，本フィルタリングの対象外とする．取得したセグメントから，動的文字列テーブルの仮想アドレスを示す DT_STRTAB と，そのサイズを示す DT_STRSZ を取得する．その後，DT_STRTAB の仮想アドレスをファイルオフセットへ変換し，動的文字列テーブルを読み出す．動的文字列テーブルには，動的リンクで使用されるシンボル名や依存ライブラリ名などが格納されているため，ソケット通信に関連する関数やライブラリの参照を簡易的に確認できる．

読み出した動的文字列テーブルを NULL 文字で分割し，各エントリが通信関連キーワードと一致するかを確認する．本実装では，socket をはじめとするソケット通信に関連するシステムコールを通信関連キーワードとして使用する．これらのキーワードのうち一つ以上が動的文字列テーブルに含まれる場合，そのバイナリをソケット通信に関与する可能性のある候補バイナリとして抽出する．抽出した候補バイナリのパスは一覧としてファイルに出力し，3.3 節で述べる通信相手の特定処理への入力とする．

本フィルタリングは，動的文字列テーブルに含まれる文字列のみを用いた簡易的な判定である．そのため，ソケット関連関数の文字列を含んでいても，実行時には該当関数を呼び出さないバイナリが候補として抽出される可能性がある．一方，独自に実装された通信処理，関数名が削除または変更されたバイナリ，静的リンク形式のバイナリ，および本実装で指定していないソケット関連関数のみを使用するバイナリは抽出できない可能性がある．これらの誤検出を含む候補バイナリについては，後続の動的解析によって実際のソケット通信の有無を確認する．

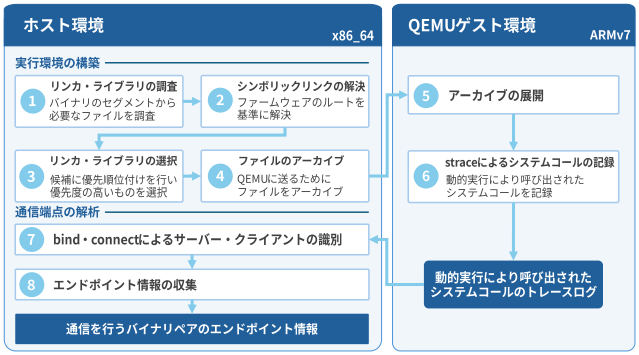


図 2　ソケット通信エンドポイントの特定処理

Fig. 2　Process for Extracting Socket Communication Endpoint Information.

3.3　通信関係候補の特定

前節で抽出した候補バイナリを実際に動作させ，実行中に発行される通信関連システムコールを観測することで，通信関係を形成する可能性のあるバイナリを特定する．処理の流れを**図 2** に示す．本処理では，候補バイナリごとに実行環境を構築し，ARMv7 の QEMU 仮想環境上で一定時間実行する．その後，実行時に発行された bind および connect システムコールを解析し，観測された通信端点を bind 側または connect 側に分類するとともに，IP アドレスおよびポート番号を取得する．

3.3.1　実行に必要なファイルの収集

候補バイナリを解析環境上で動作させるためには，バイナリ本体に加えて，対応する動的リンカおよび依存する共

© 2026 Information Processing Society of Japan

表 1 動的リンカ候補の選択基準
Table 1 Selection Criteria for Dynamic Linker Candidates.

優先度	条件
1	要求されたファイル名と完全に一致する
2	動的リンカ形式の名前を持ち，共通部分がある
3	動的リンカ形式の名前を持つ
4	要求された名前との共通部分がある
5	その他の ELF ファイル

有ライブラリが必要となる．そこで，候補バイナリが格納されているファームウェアのルートファイルシステムを特定し，ELF に含まれる動的リンク情報を基に，実行に必要なファイルを収集する．

まず，各候補バイナリの PT_INTERP セグメントから，実行時に使用される動的リンカのパスを取得する．また，PT_DYNAMIC セグメント内の DT_NEEDED エントリから，バイナリが依存する共有ライブラリ名を取得する．共有ライブラリについても同様に依存関係を解析し，実行に必要なファイルを再帰的に収集する．

ファームウェア内では，動的リンカや共有ライブラリがシンボリックリンクとして配置されている場合がある．しかし，ファームウェアのアンパック時にリンク情報が正しく復元されず，リンク先のファイルが存在しない壊れたシンボリックリンクとなる場合がある．そこで本処理では，シンボリックリンクのリンク先が有効であることを確認し，リンクを解決したうえで，リンク先が ELF ファイルであるかを判定する．

3.3.2 動的リンカおよび共有ライブラリの選択

PT_INTERP に記録されたパスから動的リンカを取得できる場合は，当該ファイルを使用する．一方，記録されたパスに対応するファイルが存在しない場合は，ファームウェア内に存在する ELF ファイルから動的リンカの候補を列挙し，各候補に優先順位を付けて使用するファイルを選択する．

動的リンカ候補の順位付けでは，候補のファイル名，要求された動的リンカ名との類似度，配置場所，およびパス長を使用する．主な選択基準を**表 1** に示す．まず，要求された動的リンカのファイル名と完全に一致する候補を最優先とする．次に，ld-，ld.so などの動的リンカに特徴的な名前を持ち，かつ要求された名前との共通部分を持つ候補を優先する．その後，動的リンカに特徴的な名前を持つ候補，要求された名前との共通部分を持つ候補，およびその他の候補の順に評価する．

共有ライブラリの選択では，DT_NEEDED に記録されたライブラリ名と一致するファイルを探索する．完全に一致するファイルが存在しない場合は，要求された名前にバージョン番号が付加されたファイルも候補とする．候補が複数存在する場合は，lib，usr/lib，その他の lib ディレクトリ，およびその他のディレクトリの順に優先する．同じ配置条件を持つ候補については，パスが短いファイルを選択する．

3.3.3 候補バイナリの実行

収集した候補バイナリ，動的リンカ，および共有ライブラリは，元のルートファイルシステムにおけるディレクトリ構造を維持したまま一時ディレクトリへ複製する．その後，収集したファイル群をアーカイブし，ARMv7 の QEMU ゲストへ転送する．この方法により，ファームウェアのルートファイルシステム全体ではなく，候補バイナリの実行に必要なファイルのみを解析環境へ配置する．

QEMU ゲスト上では，候補バイナリに対応する動的リンカを明示的に起動し，候補バイナリを引数として与える．また，収集した共有ライブラリを読み込めるように，それらの配置先を LD_LIBRARY_PATH へ設定する．実行時に不足している共有ライブラリが検出された場合は，エラーメッセージからライブラリ名を取得し，対象ファームウェアから追加のライブラリを収集した上で再実行する．本実装では，この再試行回数に上限を設ける．

候補バイナリの実行中に発行されるシステムコールは，strace を用いて記録する．子プロセスによって発行されるシステムコールも取得するため，-ff オプションを使用してプロセスごとにトレースログを出力する．また，ファイルディスクリプタに対応するソケット情報を出力するために-yy オプションを使用する．デーモンなどの終了しないバイナリによって解析処理が停止することを防ぐため，各候補バイナリには実行時間の上限を設定する．

実行後，QEMU ゲスト上に生成されたトレースログをホスト環境へ回収し，候補バイナリごとに保存する．なお，最初の execve システムコールが ENOENT によって失敗した場合は，候補バイナリを正常に実行できなかったものとして扱う．

3.3.4 エンドポイント情報の抽出

回収したトレースログから，bind および connect システムコールを抽出する．本研究では，bind を発行したバイナリを bind 側候補，connect を発行したバイナリを connect 側候補 とし，これらを通信端点の種別として記録する．

本実装では，IPv4 および IPv6 のソケットアドレスから，IP アドレスとポート番号を取得する．同一のバイナリについて，同じ通信端点の種別，IP アドレス，およびポート番号が複数回観測された場合は，重複した結果を一つに集約する．最終的に，バイナリのパス，通信端点の種別，IP アドレス，およびポート番号をエンドポイント情報として出力する．

出力されたエンドポイント情報において，同一のファームウェアに含まれ，同じポート番号を使用する bind 側候補と connect 側候補を，通信関係候補として抽出する．bind 側候補の IP アドレスが 0.0.0.0 または:: である場合は，

© 2026 Information Processing Society of Japan

表 2　評価環境
Table 2　Experimental Environment.

項目	ホスト環境	QEMU ゲスト環境
OS	Ubuntu 22.04.5 LTS	Debian GNU/Linux 12
CPU	AMD Ryzen 9 7900X	Cortex-A15，3 vCPU
メモリ	124 GiB	1536 MiB
アーキテクチャ	x86_64	ARMv7 little-endian

表 3　テストファームウェアに対する性能評価
Table 3　Performance Evaluation on the Test Firmware.

処理	平均時間 [s]	標準偏差 [s]	平均 RSS [MiB]
アンパック	0.730	0.129	136.21
フィルタリング	0.232	0.001	18.77
通信端点情報の抽出	102.617	0.612	39.12
動的テイント解析	8.313	0.141	37.53

すべてのインタフェースで待ち受けているものとして扱う．本処理により抽出した通信関係候補を，動的テイント解析の対象とする．

3.4　動的テイント解析

事前解析によって通信関係の候補として抽出されたバイナリの組に対して，動的テイント解析を適用する．本研究で使用するテイント解析エンジンには，複数のバイナリを並行して解析できることに加え，ソケット通信を介してプロセス間を伝播する情報フローを追跡できることが求められる．そこで，本解析部には，著者らの既発表研究 [16] で提案した動的テイント解析手法を用いる．動的テイント解析では，対象となるバイナリの組を選択し，両方のバイナリを解析環境上で実行する．各プロセス内部におけるデータの伝播に加え，ソケット通信によってプロセス間を移動するデータについてもテイント情報を引き継ぐことで，複数プロセスにまたがる情報フローを追跡する．

4.　評価

本章では，提案手法の有効性を，テストファームウェアおよび実世界のファームウェアデータセットを用いて評価する．評価環境の詳細を**表 2** に示す．

4.1　テストファームウェアによる性能評価

OpenWrt[17] を基盤とし，同一機器内でソケット通信を行うサーバー・クライアントバイナリ 1 組を組み込んだテストファームウェアを作成し，提案手法の処理性能および解析対象の絞り込み効果を評価した．性能評価では，提案手法をアンパック，ソケット通信バイナリのフィルタリング，エンドポイント情報の抽出，および動的テイント解析の 4 段階に分け，各処理に要する実行時間とメモリ使用量を測定した．各処理を 10 回実行し，実行時間については平均値と標準偏差を，メモリ使用量については各試行における RSS の最大値の平均値を算出した．各処理の測定結果を**表 3** に示す．また，各処理段階における解析対象数を計測し，事前解析によって動的テイント解析の対象がどの程度絞り込まれるかを確認した．その結果を**表 4** に示す．

表 4　各処理段階における解析対象の絞り込み結果
Table 4　Reduction in the Number of Analysis Targets at Each Processing Stage.

処理結果	対象数
静的フィルタリング	
通常ファイル	1,102 ファイル
ELF ファイル	146 バイナリ
共有ライブラリ除外後	110 バイナリ
動的文字列テーブルを取得可能	43 バイナリ
通信関連キーワードに一致	19 バイナリ
エンドポイント情報の抽出	
`strace` による実行成功	19 バイナリ
エンドポイント情報を取得	3 バイナリ
通信関係候補を抽出	1 組
動的テイント解析	
動的テイント解析を完了	1 組

フィルタリングの結果，直接の解析対象となり得る 110 個の ELF バイナリから，通信関連キーワードを含む 19 個のバイナリまで候補を絞り込むことができた．さらに，動的解析によって 3 個のバイナリから通信端点情報を取得し，このうち 2 個のバイナリを対応付け，テストファームウェアに組み込んだ通信ペアに対応する 1 組を通信関係候補として抽出した．抽出した 1 組に対して動的テイント解析を適用した結果，解析は正常に完了した．

4.2　実世界データセットによる予備的評価

本節では，実世界のファームウェアデータセットを用いて，提案手法の実ファームウェアに対する適用可能性を予備的に評価する．評価には，Wu らが収集したファームウェアデータセット [18] を使用する．本研究の解析対象に合わせ，データセットから ARMv7 アーキテクチャ向けのファームウェアを 500 件抽出し，提案手法の各段階の効果を手動解析を含めて調査した．

使用したデータセットには，CPU アーキテクチャを直接示すメタデータが含まれていない．そこで，ファームウェアの URL および製品名に ARMv7 向けプラットフォーム名が含まれるものを候補として抽出した．さらに，SDK，ソースコード，ブートローダーなど，通常のルートファイルシステムを含まない配布物を除外した．

ただし，本選定方法はファイル名や URL の命名規則に依存するため，OpenWrt など，アーキテクチャ情報が明示された配布物が選択されやすいという偏りを含む．したがって，本評価はデータセット全体を代表するものではなく，提案手法の実世界ファームウェアに対する適用可能性

© 2026 Information Processing Society of Japan

表 5　候補バイナリに対する動的通信解析の結果
Table 5　Results of Dynamic Communication Analysis for Candidate Binaries.

指標	バイナリ数
解析対象	8,571
bind/connect を観測	5,034
UNIX ドメインソケットを使用	4,575
IPv4/IPv6 ソケットを使用	459
IPv4/IPv6 ソケットで bind を観測	459
IPv4/IPv6 ソケットで connect を観測	0

を確認する予備的評価として位置付ける．

4.2.1　アンパックおよびフィルタリング結果

アンパックの結果，500 件中 468 件から解析可能なルートファイルシステムを抽出できた．取得した 468 件のルートファイルシステムには合計 448,540 個の通常ファイルと 73,408 個の ELF バイナリが含まれていた．これらに対して 3.2 節のフィルタリングを適用した結果，8,571 個のバイナリを候補として抽出した．

提案手法のフィルタリングによって除外されたバイナリの中にソケット通信を行うバイナリが存在するかを調べたところ，4 個のバイナリが除外されていた．2 個のバイナリは静的リンクバイナリであったため，提案手法では解析対象外である．

4.2.2　動的通信解析の結果

フィルタリングによって抽出した 8,571 個の候補バイナリを，3.3 節で述べた手法により 1 件ずつ実行した．各バイナリは，引数および設定ファイルを与えずに 5 秒間実行し，実行中に発行された通信関連システムコールを観測した．動的通信解析の結果を**表 5** に示す．

8,571 個すべての候補バイナリについて解析結果を取得した．このうち 5,034 個，すなわち 58.7%のバイナリから，bind または connect システムコールを観測した．観測した 5,034 個のうち，4,575 個は AF_UNIX を使用する UNIX ドメインソケットであり，459 個は AF_INET または AF_INET6 を使用する IPv4/IPv6 ソケットであった．

IPv4/IPv6 ソケットを使用した 459 個のバイナリは，すべて bind を実行しており，connect を実行したバイナリは観測されなかった．このため，bind 側候補と対応付ける connect 側候補が得られず，動的テイント解析の対象となる通信関係候補は 0 組であった．

connect が観測されなかった要因として，引数および設定ファイルを与えない 5 秒間の単独実行では，接続先の決定や通信処理の開始に必要な実行条件を満たせなかったことが考えられる．また，3,537 個の候補バイナリについては，設定した実行時間内に bind または connect へ到達しなかった．ただし，これらのバイナリには，コマンドライン引数，設定ファイル，ネットワーク状態，または他のプロセスを必要とするものが含まれる可能性があるため，すべてをフィルタリングの誤検出として扱うことはできない．

なお，本実装では UNIX ドメインソケットのシステムコール自体は観測できるものの，通信端点の抽出およびバイナリ間の対応付けは，IPv4/IPv6 の IP アドレスとポート番号を対象としている．そのため，UNIX ドメインソケットを用いるバイナリについては，通信相手の特定および動的テイント解析対象の生成を行っていない．

5.　今後の課題

第 4 章の評価では，テストファームウェアに対して，通信候補バイナリの抽出，通信関係候補の抽出，および動的テイント解析までの一連の処理を完了できた．また，実世界ファームウェア 468 件に提案手法を適用し，73,408 個の ELF バイナリから 8,571 個の通信候補バイナリを抽出した．このうち 5,034 個のバイナリから実行時の bind または connect を観測できたことから，静的フィルタリングと QEMU 上での動的解析を組み合わせることで，詳細解析の対象となる通信関連バイナリを段階的に絞り込めることを確認した．

テストファームウェアでは通信関係を抽出できた一方，実世界ファームウェアでは connect が観測されなかった．このことから，現行の単独実行では，connect の発行に必要な実行条件を十分に再現できていないことが主な要因であると考えられる．今後は，起動スクリプトや設定ファイルを解析し，コマンドライン引数，環境変数，作業ディレクトリ，ネットワーク設定，および依存サービスを推定する必要がある．さらに，関連する複数のプロセスを実際の起動順序に従って協調動作させることで，単独実行では到達できなかった通信処理を誘発する必要がある．

また，bind または connect を観測した 5,034 個のバイナリのうち，4,575 個，すなわち 90.9%が UNIX ドメインソケットを使用していた．現状の提案手法では，IPv4/IPv6 ソケットを用いる通信関係のみを後続の解析候補としているため，UNIX ドメインソケットを介した通信関係は解析対象に含まれない．この結果から，同一機器内のプロセス間通信を幅広く解析するためには，IP ソケットだけでなく，UNIX ドメインソケットを利用するバイナリの組についても通信関係候補として抽出する必要がある．

6.　まとめ

本研究では，IoT ファームウェア内の複数プロセス間を伝播する情報フローを効率的に解析することを目的として，ソケット通信に着目したファームウェア解析フレームワークを提案した．提案手法では，ファームウェアをアンパックした後，バイナリの動的文字列テーブルを用いて通信候補を抽出し，QEMU 上での実行時に観測した bind および connect の情報から通信関係候補を抽出する．その後，抽出した通信関係候補に対してのみ動的テイント解析を適用

© 2026 Information Processing Society of Japan

することで，ファームウェア全体を詳細に解析する場合と比較して，解析対象および解析負荷の削減を図った．

テストファームウェアを用いた評価では，110 個の ELF バイナリから 19 個の通信候補バイナリまで対象を絞り込み，通信関係候補を 1 組抽出した上で，ソケット通信を介した動的テイント解析を完了できた．また，実世界ファームウェア 500 件を用いた予備的評価では，468 件から解析可能なルートファイルシステムを取得し，73,408 個の ELF バイナリから 8,571 個の通信候補バイナリを抽出した．さらに，5,034 個のバイナリから実行時のソケット操作を観測し，提案した事前解析を実世界ファームウェアへ適用可能であることを確認した．

一方，実世界ファームウェアでは，IPv4/IPv6 ソケットに対する `connect` を観測できず，通信関係候補の抽出には至らなかった．また，観測した通信の多くが UNIX ドメインソケットであったことから，IPv4/IPv6 ソケットのみを対象とする現状の手法では，同一機器内の通信関係を十分に網羅できないことが明らかとなった．今後は，バイナリの実行条件を再現する機構を導入するとともに，IP ソケットに加えて UNIX ドメインソケットを利用するバイナリの組も通信関係候補として抽出することで，実世界ファームウェアに対する解析の適用範囲を拡大する．

謝辞　本研究は JSPS 科研費 JP25K15111 の助成を受けたものである．

参考文献

[1] Antonakakis, M., April, T., Bailey, M., Bernhard, M., Bursztein, E., Cochran, J., Durumeric, Z., Halderman, J. A., Invernizzi, L., Kallitsis, M. et al.: Understanding the Mirai botnet, *26th USENIX Security Symposium (USENIX Security 17)*, pp. 1093–1110 (2017).

[2] Federal Bureau of Investigation et al.: People's Republic of China-Linked Actors Compromise Routers and IoT Devices for Botnet Operations, CISA Advisory AA24-270A (2024).

[3] Ul Haq, S., Singh, Y., Sharma, A., Gupta, R. and Gupta, D.: A survey on IoT & embedded device firmware security: architecture, extraction techniques, and vulnerability analysis frameworks, *Discover Internet of Things*, Vol. 3, No. 1, p. 17 (2023).

[4] Cheng, K., Li, Q., Wang, L., Chen, Q., Zheng, Y., Sun, L. and Liang, Z.: DTaint: Detecting the Taint-Style Vulnerability in Embedded Device Firmware, *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 430–441 (2018).

[5] Chen, D. D., Woo, M., Brumley, D. and Egele, M.: Towards Automated Dynamic Analysis for Linux-based Embedded Firmware, *Network and Distributed System Security Symposium* (2016).

[6] Redini, N., Machiry, A., Wang, R., Spensky, C., Continella, A., Shoshitaishvili, Y., Kruegel, C. and Vigna, G.: Karonte: Detecting insecure multi-binary interactions in embedded firmware, *2020 IEEE Symposium on Security and Privacy (SP)*, pp. 1544–1561 (2020).

[7] Zhao, J., Li, Y., Zou, Y., Liang, Z., Xiao, Y., Li, Y., Peng, B., Zhong, N., Wang, X., Wang, W. and Huo, W.: Leveraging Semantic Relations in Code and Data to Enhance Taint Analysis of Embedded Systems, *33rd USENIX Security Symposium (USENIX Security 24)*, pp. 7067–7084 (2024).

[8] Ji, Y., Lee, S., Fazzini, M., Allen, J., Downing, E., Kim, T., Orso, A. and Lee, W.: Enabling Refinable Cross-Host Attack Investigation with Efficient Data Flow Tagging and Tracking, *27th USENIX Security Symposium (USENIX Security 18)*, pp. 1705–1722 (2018).

[9] Yang, M., Zhou, X., Liu, D., Zhou, L. and Tang, Y.: Enhancing IoT Security: A Full-System Simulation Dynamic Taint Analysis Framework for Firmware, *2023 3rd International Conference on Electronic Information Engineering and Computer (EIECT)*, pp. 381–388 (2023).

[10] Enck, W., Gilbert, P., Han, S., Tendulkar, V., Chun, B.-G., Cox, L. P., Jung, J., McDaniel, P. and Sheth, A. N.: TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones, *ACM Trans. Comput. Syst.*, Vol. 32, No. 2, pp. 5:1–5:29 (2014).

[11] Cheng, K., Zheng, Y., Liu, T., Guan, L., Liu, P., Li, H., Zhu, H., Ye, K. and Sun, L.: Detecting Vulnerabilities in Linux-Based Embedded Firmware with SSE-Based On-Demand Alias Analysis, *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 360–372 (2023).

[12] Chen, L., Wang, Y., Cai, Q., Zhan, Y., Hu, H., Linghu, J., Hou, Q., Zhang, C., Duan, H. and Xue, Z.: Sharing More and Checking Less: Leveraging Common Input Keywords to Detect Bugs in Embedded Systems, *30th USENIX Security Symposium (USENIX Security 21)*, pp. 303–319 (2021).

[13] Zaddach, J., Bruno, L., Francillon, A. and Balzarotti, D.: AVATAR: A framework to support dynamic security analysis of embedded systems, *NDSS 2014, Network and Distributed System Security Symposium, 23-26 February 2014, San Diego, USA* (ISOC, ed.) (2014).

[14] Fu, X. and Cai, H.: FlowDist: Multi-Staged Refinement-Based Dynamic Information Flow Analysis for Distributed Software Systems, *30th USENIX Security Symposium (USENIX Security 21)*, pp. 2093–2110 (2021).

[15] Izzillo, A., Lazzeretti, R. and Coppa, E.: STAFF: Stateful Taint-Assisted Full-system Firmware Fuzzing, *Computers & Security*, Vol. 170, p. 105003 (2026).

[16] 東　政澄，掛井将平，齋藤彰一：ソケット通信によるデータフローを考慮した IoT デバイス向け動的テイント解析システムの開発，コンピュータセキュリティシンポジウム 2025 論文集，pp. 969–976 (2025).

[17] OpenWrt Project: OpenWrt, `https://openwrt.org/`. Accessed: 2026-08-02.

[18] Wu, Y., Wang, J., Wang, Y., Zhai, S., Li, Z., He, Y., Sun, K., Li, Q. and Zhang, N.: Your Firmware Has Arrived: A Study of Firmware Update Vulnerabilities, *USENIX Security Symposium* (2024).

© 2026 Information Processing Society of Japan